

Reviewer Pack — Minimal Representation Length of Affine Quotients and Circui...

Entry 28e7275f53fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 01:34:01 UTC

Minimal Representation Length of Affine Quotients and Circuit Monotone Complexity

Entry ID: 28e7275f53fe **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 01:34:01 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Boolean Circuits

Statement:

For every boolean circuit C with n variables, the minimal representation length required to describe the affine quotient of C 's output polynomial modulo 2 over a finite field is $O(n \log n)$, where the representation length is measured in terms of bits needed for the representation of the coefficients and exponents of the quotient.

Rationale (proposer's reasoning):

Affine geometry has been applied to circuit complexity via the study of representation theory, but there is little work on the direct impact of affine quotient properties on circuit monotone complexity. This conjecture proposes a specific connection that could reveal new insights into the structure of circuits and their complexity.

Taxonomy category: Affine_Quotient (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dc325cfce181e07e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$ and at least 80% of the circuits, the representation length of the affine quotient $q(C)$ is $O(n \log n)$, measured in bits, with no seed producing a representation length greater than $1.5 * n \log n$. The conjecture is falsified if any seed produces a representation length exceeding $1.5 * n \log n$ or if less than 80% of the circuits meet the $O(n \log n)$ condition.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine geometry" AND "Boolean circuits" AND minimal representation length" - "circuit monotone complexity" AND affine quotient AND bit representation" - "exponent coefficient bits" IN BOOLEAN CIRCUITS AND affine geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    def generate_random_circuit(n):
        circuit = []
        for _ in range(2 * n - 1):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(random.randint(1, n))]
            circuit.append((gate_type, inputs))
        return circuit

    def evaluate_circuit(circuit, input_values):
        stack = []
        for gate_type, inputs in reversed(circuit):
            if gate_type == 'AND':
                result = all(stack.pop() for _ in inputs)
            elif gate_type == 'OR':
                result = any(stack.pop() for _ in inputs)
            else:
                raise ValueError(f"Invalid gate type: {gate_type}")
            stack.append(result)
        return stack[0]

    def polynomial_representation_length(p):
        length = 0
        for coeff, exp in p.items():
            length += len(bin(coeff)) + sum(len(bin(e)) for e in exp)
        return length

    def affine_quotient(circuit):
        input_values = [random.randint(0, 1) for _ in range(n)]
```

```

    output = evaluate_circuit(circuit, input_values)
    p = {output: tuple(input_values)}
    return p

n = random.choice([5, 10, 15, 20, 30, 40])
circuit = generate_random_circuit(n)
q = affine_quotient(circuit)
representation_length = polynomial_representation_length(q)

metric_value = representation_length
instances_tested = 1
n_max = n
conjecture_holds = representation_length <= 1.5 * n * math.log2(n)
counterexample = "" if conjecture_holds else f"n={n}, length={representation_length}"

return {
    "metric_name": "Representation Length",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
                           len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={n_max}<{result['n_max']}, length>{1.5 * result['metric_value']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b5e4363d.py", line 75, in <module>
    trial_result = run_trial(seed)
    ~~~~~^~~~~~

```


2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/28e7275f53fe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/28e7275f53fe.tar.gz` (if generated)